Pipeline traces: assignment sections 8.2 - 8.4

Cycle-by-cycle traces of the generated circuit (stage occupancy from the pipeline registers, valid bits, hazard-unit outputs, forwarding selects and the architectural writes seen at WB / MEM). Cycle 0 is the first cycle after reset. Notation: **x** before an instruction = fetched but squashed (V = 0); '-' = bubble / empty; **V** column = valid bits of IF/ID, ID/EX, EX/MEM, MEM/WB; forwarding is reported for the instruction in EX (A/B operand, source register). Writes: Rn<-v at WB, RAM/peripheral writes at MEM (once per store). The same programs were also run in Logisim 2.7.1 and the STALL, FLUSH, valid-bit and write-strobe probes matched cycle for cycle.

8.2 Load-use dependence (exactly one stall)

```
LDI R2, 0x20
LDI R4, 0x2A
ST  R4, [R2]
LDI R3, 3
LD  R1, [R2]
ADD R3, R1
ST  R3, [R2]
NOP
NOP
HALT
```

Cyc	IF	ID	EX	MEM	WB	V	Stall	Flush	Forwarding (instr in EX)	Architectural writes	VALVE
0	00:LDI R2, 0x20	-	-	-	-	0000	0	0	-	-	0
1	01:LDI R4, 0x2A	00:LDI R2, 0x20	-	-	-	1000	0	0	-	-	0
2	02:ST R4, [R2]	01:LDI R4, 0x2A	00:LDI R2, 0x20	-	-	1100	0	0	-	-	0
3	03:LDI R3, 3	02:ST R4, [R2]	01:LDI R4, 0x2A	00:LDI R2, 0x20	-	1110	0	0	-	-	0
4	04:LD R1, [R2]	03:LDI R3, 3	02:ST R4, [R2]	01:LDI R4, 0x2A	00:LDI R2, 0x20	1111	0	0	A<-EX/MEM B<-MEM/WB	R2<-0x20	0
5	05:ADD R3, R1	04:LD R1, [R2]	03:LDI R3, 3	02:ST R4, [R2]	01:LDI R4, 0x2A	1111	0	0	-	R4<-0x2A; RAM[0x20]<-0x2A	0
6	06:ST R3, [R2]	05:ADD R3, R1	04:LD R1, [R2]	03:LDI R3, 3	02:ST R4, [R2]	1111	1	0	-	-	0
7	06:ST R3, [R2]	05:ADD R3, R1	-	04:LD R1, [R2]	03:LDI R3, 3	1011	0	0	-	R3<-0x03	0
8	07:NOP	06:ST R3, [R2]	05:ADD R3, R1	-	04:LD R1, [R2]	1101	0	0	B<-MEM/WB	R1<-0x2A	0
9	08:NOP	07:NOP	06:ST R3, [R2]	05:ADD R3, R1	-	1110	0	0	A<-EX/MEM	-	0
10	09:HALT	08:NOP	07:NOP	06:ST R3, [R2]	05:ADD R3, R1	1111	0	0	-	R3<-0x2D; RAM[0x20]<-0x2D	0
11	0A:NOP(0)	09:HALT	08:NOP	07:NOP	06:ST R3, [R2]	1111	0	0	-	-	0
12	0B:NOP(0)	0A:NOP(0)	09:HALT	08:NOP	07:NOP	1111	0	1	-	-	0
13	(halted)	x 0B:NOP(0)	-	09:HALT	08:NOP	0011	0	0	-	-	0
14	(halted)	x 0B:NOP(0)	-	-	09:HALT	0001	0	0	-	-	0
15	(halted)	x 0B:NOP(0)	-	-	-	0000	0	0	-	-	0

What the trace shows

- LD R1,[R2] is in EX at cycle 6 while ADD R3,R1 is in ID: hazard unit raises STALL (usesB of ADD matches LD dest). PC and IF/ID freeze (IF shows 06 twice, ID keeps ADD) and ID/EX receives V = 0.
- Cycle 7: bubble in EX (V of ID/EX = 0), LD advances to MEM, nothing is written by the bubble. Exactly one stall cycle in the whole trace.
- Cycle 8: ADD enters EX; operand B (R1) is forwarded from MEM/WB (load data), operand A (R3) came through the register-file WB bypass. Result 0x2D is written to R3 at cycle 10.
- Each of the two stores commits exactly once: RAM[0x20]<-0x2A at cycle 5 and RAM[0x20]<-0x2D at cycle 10 (the second ST R3,[R2] needs R3 from EX/MEM, no stale data).

Orange rows: STALL. Purple rows: FLUSH (branch/jump redirect or HALT).

8.3 Youngest-producer forwarding to a store (no stale store data)

```
LDI R1, 10
LDI R2, 2
LDI R3, 3
LDI R4, 0x30
ADD R1, R2
SUB R1, R3
ST  R1, [R4]
NOP
NOP
HALT
```

Cyc	IF	ID	EX	MEM	WB	V	Stall	Flush	Forwarding (instr in EX)	Architectural writes	VALVE
0	00:LDI R1, 10	-	-	-	-	0000	0	0	-	-	0
1	01:LDI R2, 2	00:LDI R1, 10	-	-	-	1000	0	0	-	-	0
2	02:LDI R3, 3	01:LDI R2, 2	00:LDI R1, 10	-	-	1100	0	0	-	-	0
3	03:LDI R4, 0x30	02:LDI R3, 3	01:LDI R2, 2	00:LDI R1, 10	-	1110	0	0	-	-	0
4	04:ADD R1, R2	03:LDI R4, 0x30	02:LDI R3, 3	01:LDI R2, 2	00:LDI R1, 10	1111	0	0	-	R1<-0x0A	0
5	05:SUB R1, R3	04:ADD R1, R2	03:LDI R4, 0x30	02:LDI R3, 3	01:LDI R2, 2	1111	0	0	-	R2<-0x02	0
6	06:ST R1, [R4]	05:SUB R1, R3	04:ADD R1, R2	03:LDI R4, 0x30	02:LDI R3, 3	1111	0	0	-	R3<-0x03	0
7	07:NOP	06:ST R1, [R4]	05:SUB R1, R3	04:ADD R1, R2	03:LDI R4, 0x30	1111	0	0	A<-EX/MEM	R4<-0x30	0
8	08:NOP	07:NOP	06:ST R1, [R4]	05:SUB R1, R3	04:ADD R1, R2	1111	0	0	A<-EX/MEM	R1<-0x0C	0
9	09:HALT	08:NOP	07:NOP	06:ST R1, [R4]	05:SUB R1, R3	1111	0	0	-	R1<-0x09; RAM[0x30]<-0x09	0
10	0A:NOP(0)	09:HALT	08:NOP	07:NOP	06:ST R1, [R4]	1111	0	0	-	-	0
11	0B:NOP(0)	0A:NOP(0)	09:HALT	08:NOP	07:NOP	1111	0	1	-	-	0
12	(halted)	x 0B:NOP(0)	-	09:HALT	08:NOP	0011	0	0	-	-	0
13	(halted)	x 0B:NOP(0)	-	-	09:HALT	0001	0	0	-	-	0
14	(halted)	x 0B:NOP(0)	-	-	-	0000	0	0	-	-	0

What the trace shows

• Cycle 8: ST R1,[R4] is in EX. R1 has two older producers in flight: SUB R1,R3 in MEM (EX/MEM) and ADD R1,R2 in WB (MEM/WB). The forwarding unit selects EX/MEM (A<-EX/MEM): the **youngest** producer wins.

• Cycle 9: RAM[0x30] is written once with 0x09 (= (10+2)-3), not 0x0C (the older ADD result) and not 0x0A (the stale register-file value). No stall is needed (all producers are ALU ops).

Orange rows: STALL. Purple rows: FLUSH (branch/jump redirect or HALT).

8.4 Taken branch with wrong-path actuator write

```
    LDI R7, 0xF4
    LDI R2, 1
    LDI R1, 6
    LDI R3, 9
    LDI R4, 5
    SUB R1, R1
    BEQZ R1, target
    ST  R2, [R7]
    ADD R3, R4
target:
    NOP
    NOP
    NOP
    HALT
```

Cyc	IF	ID	EX	MEM	WB	V	Stall	Flush	Forwarding (instr in EX)	Architectural writes	VALVE
0	00:LDI R7, 0xF4	-	-	-	-	0000	0	0	-	-	0
1	01:LDI R2, 1	00:LDI R7, 0xF4	-	-	-	1000	0	0	-	-	0
2	02:LDI R1, 6	01:LDI R2, 1	00:LDI R7, 0xF4	-	-	1100	0	0	-	-	0
3	03:LDI R3, 9	02:LDI R1, 6	01:LDI R2, 1	00:LDI R7, 0xF4	-	1110	0	0	-	-	0
4	04:LDI R4, 5	03:LDI R3, 9	02:LDI R1, 6	01:LDI R2, 1	00:LDI R7, 0xF4	1111	0	0	-	R7<-0xF4	0
5	05:SUB R1, R1	04:LDI R4, 5	03:LDI R3, 9	02:LDI R1, 6	01:LDI R2, 1	1111	0	0	-	R2<-0x01	0
6	06:BEQZ R1, target	05:SUB R1, R1	04:LDI R4, 5	03:LDI R3, 9	02:LDI R1, 6	1111	0	0	-	R1<-0x06	0
7	07:ST R2, [R7]	06:BEQZ R1, target	05:SUB R1, R1	04:LDI R4, 5	03:LDI R3, 9	1111	0	0	-	R3<-0x09	0
8	08:ADD R3, R4	07:ST R2, [R7]	06:BEQZ R1, target	05:SUB R1, R1	04:LDI R4, 5	1111	0	1	A<-EX/MEM	R4<-0x05	0
9	09:NOP	x 08:ADD R3, R4	-	06:BEQZ R1, target	05:SUB R1, R1	0011	0	0	-	R1<-0x00	0
10	0A:NOP	09:NOP	-	-	06:BEQZ R1, target	1001	0	0	-	-	0
11	0B:NOP	0A:NOP	09:NOP	-	-	1100	0	0	-	-	0
12	0C:HALT	0B:NOP	0A:NOP	09:NOP	-	1110	0	0	-	-	0
13	0D:NOP(0)	0C:HALT	0B:NOP	0A:NOP	09:NOP	1111	0	0	-	-	0
14	0E:NOP(0)	0D:NOP(0)	0C:HALT	0B:NOP	0A:NOP	1111	0	1	-	-	0
15	(halted)	x 0E:NOP(0)	-	0C:HALT	0B:NOP	0011	0	0	-	-	0
16	(halted)	x 0E:NOP(0)	-	-	0C:HALT	0001	0	0	-	-	0
17	(halted)	x 0E:NOP(0)	-	-	-	0000	0	0	-	-	0

What the trace shows

• SUB R1,R1 makes R1 = 0; BEQZ in EX at cycle 8 is taken: FLUSH = 1. The wrong-path ST R2,[R7] (in ID) and ADD R3,R4 (being fetched) are discarded: at cycle 9 ID/EX is invalid (EX shows '-') and the ADD in IF/ID is marked x (V = 0). Exactly two wrong-path instructions.

• PC <- target (0x09) at the same edge. The squashed ST never reaches MEM: no ACT write appears in the Writes column, VALVE_IN stays 0 in every cycle (column VALVE).

• The branch itself (V = 1) continues to MEM and WB with regWrite = memWrite = 0, so it writes nothing.

Orange rows: STALL. Purple rows: FLUSH (branch/jump redirect or HALT).